

MiniLexer

Um analisador léxico para a MiniLang construído sobre seis Expressões Regulares e seus AFN ϵ de Thompson

Andrey Lourival Andrade Garcia
Igor Cecim Vilhena

Python 3 · re · pytest · JFLAP
30/09/2026

Transformar código-fonte em tokens válidos

ENTRADA

Código MiniLang

Arquivo .min ou uma linha digitada no terminal, incluindo entrada vazia.

PROCESSAMENTO

MiniLexer

Isola cada lexema e valida o candidato com `re.fullmatch` contra a ER correspondente.

SAÍDA

Tokens ou erro

Lista de tokens com tipo, lexema, linha e coluna, ou um erro léxico com o motivo.

```
float fator = 1.5e3;
```



```
TYPE_FLOAT · IDENTIFIER · ASSIGN · SCIENTIFIC · SEMICOLON
```

Módulos em Python

Módulo	Responsabilidade
automata.py	6 regex, AFNε de Thompson, simulador
patterns.py	Palavras reservadas, operadores
tokens.py	Tipos de token
lexer.py	MiniLexer e erros léxicos
main.py	Interface de linha de comando

Ordem de tokenização

1. espaço em branco (descartado)
2. comentário // (descartado)
3. string — ER-05
4. operador composto == != <= >=
5. científico — ER-04
6. decimal — ER-03
7. horário — ER-06
8. inteiro — ER-02
9. identificador / reservada — ER-01
10. operador simples, delimitador
11. erro léxico

Notação formal e sintaxe Python

Operador	Formal	Python	Significado
União	$r \mid s$	<code>r s</code>	$L(r) \cup L(s)$
Concatenação	rs	<code>rs</code>	justaposição
Fecho de Kleene	r^*	<code>r*</code>	zero ou mais; inclui ϵ
Fecho positivo	$r r^*$	<code>r+</code>	uma ou mais
Opcional	$(r \mid \epsilon)$	<code>r?</code>	zero ou uma
Classe	$(0 1 \dots 9)$	<code>[0-9]</code>	abreviação da união
Ponto literal	$P = \{.\}$	<code>\.</code>	escape do metacaractere

Precedência

`* + ? >` concatenação `>` união

Âncoras

`re.fullmatch` exige que a cadeia inteira pertença a L ; não é símbolo do alfabeto.

Não usamos

`\d \w` . retroreferência, lookahead

Seis Expressões Regulares

ER / token	ER formal	Sintaxe no código (re.fullmatch)
ER-01 IDENTIFIER	$L(A \mid _AA^*)^*$	<code>[A-Za-z]([A-Za-z0-9] _[A-Za-z0-9]+)^*</code>
ER-02 INTEGER	$0 \mid ND^*$	<code>(0 [1-9][0-9]^*)</code>
ER-03 DECIMAL	$(0 \mid ND^*) P D D^*$	<code>(0 [1-9][0-9]^*)\.[0-9]^+</code>
ER-04 SCIENTIFIC	$(0 \mid ND^*)(PDD^* \epsilon)(e \mid E)(+ \mid - \mid \epsilon)DD^*$	<code>(0 [1-9][0-9]^*)(\.[0-9]^+)?[eE][+-]?[0-9]^+</code>
ER-05 STRING	<code>" C* "</code>	<code>"[A-Za-z0-9 _.,!?:;+*/=<>-]^*"</code>
ER-06 TIME	$((0 \mid 1)D \mid 2(0 \mid 1 \mid 2 \mid 3)) : (0 \mid \dots \mid 5) D$	<code>([01][0-9] 2[0-3]):[0-5][0-9]</code>

L = letras ASCII · D = {0,...,9} · N = {1,...,9} · A = L ∪ D · P = {.,} · C = conteúdo permitido da string

Identificador estruturado

Alfabeto Σ

$\Sigma = L \cup D \cup \{_ \}$ · $L = \{A, \dots, Z, a, \dots, z\}$ · $D = \{0, \dots, 9\}$ · $A = L \cup D$

Linguagem L

Começa com letra; continua com letras ou dígitos; `_` só como separador interno, sempre seguido de pelo menos um alfanumérico.

ER formal

$L(A \mid _AA^*)^*$

Sintaxe no código (REGEX_ER01)

$[A-Za-z]([A-Za-z0-9] \mid _[A-Za-z0-9]^+)^*$

Equivalência

$[A-Za-z] \equiv L$ · $[A-Za-z0-9] \equiv A$ · $[A-Za-z0-9]^+ \equiv AA^*$ · $\mid \equiv \text{união}$ · $()^* \equiv \text{fecho de Kleene}$

Aceitas

a nome aluno1 valor_total mediaFinal2
a1_b2

Rejeitadas

ϵ 1aluno _nome nome_ nome__aluno
nome-aluno

Casos-limite: a (menor cadeia aceita), ϵ , nome_ e nome__aluno.

AFN ϵ de Thompson

28 estados · 35 transições (27 ϵ , 8 rotuladas) · inicial q0
· final q27

Inteiro sem zero à esquerda

Alfabeto Σ

$\Sigma = D = \{0, \dots, 9\} \cdot N = \{1, \dots, 9\}$

Linguagem L

O zero sozinho, ou um dígito de 1 a 9 seguido de zero ou mais dígitos.
Nenhum zero à esquerda.

ER formal

0 | ND*

Sintaxe no código (REGEX_ER02)

(0|[1-9][0-9]*)

Equivalência

$[1-9] \equiv N \cdot [0-9]^* \equiv D^* \cdot | \equiv \text{união} \cdot \text{a união separa os casos } \{0\} \text{ e } ND^*, \text{ por isso } D^* \text{ sozinho aceitaria } 007$

Aceitas

0 1 7 9 10 20 2026 999

Rejeitadas

ϵ 00 01 007 0123 -1 1.5

Casos-limite: 0 (aceito) e 00 (rejeitado). O sinal de - é o token MINUS.

AFNε de Thompson

10 estados · 12 transições (9 ϵ , 3 rotuladas) · inicial q0 · final q1

Número decimal

Alfabeto Σ

$\Sigma = D \cup P \cdot D = \{0, \dots, 9\} \cdot N = \{1, \dots, 9\} \cdot P = \{.\}$

Linguagem L

Parte inteira válida da ER-02, um ponto obrigatório e pelo menos um dígito depois do ponto.

ER formal

$(0 \mid ND^*) P D D^*$

Sintaxe no código (REGEX_ER03)

$(0 \mid [1-9][0-9]^*) \backslash . [0-9]^+$

Equivalência

$\backslash . \equiv P$ (o escape torna o ponto literal) · $[0-9]^+ \equiv DD^*$ · $(0 \mid [1-9][0-9]^*)$ reaproveita a ER-02

Aceitas

0.5 0.50 1.5 9.5 10.25 2026.75

Rejeitadas

00.5 01.5 .5 5. 1.2.3 -9.5

Casos-limite: 0.5 (menor parte inteira), 5. e .5 (falta um dos lados do ponto).

AFNε de Thompson

18 estados · 22 transições (16 ε, 6 rotuladas) · inicial q0
· final q17

Notação científica

Alfabeto Σ

$\Sigma = D \cup \{., e, E, +, -\} \cdot D = \{0, \dots, 9\} \cdot N = \{1, \dots, 9\} \cdot P = \{.\}$

Linguagem L

Mantissa (inteiro ou decimal válido), e ou E obrigatório, sinal opcional e expoente com pelo menos um dígito.

ER formal

$(0|ND^*)(PDD^*|\epsilon)(e|E)(+|-|\epsilon)DD^*$

Sintaxe no código (REGEX_ER04)

$(0|[1-9][0-9]^*)(\.[0-9]^+)?[eE][+-]?[0-9]^+$

Equivalência

$(\.[0-9]^+)? \equiv (PDD^*|\epsilon) \cdot [eE] \equiv (e|E) \cdot [+|-]? \equiv (+|-|\epsilon) \cdot [0-9]^+ \equiv DD^*$

Aceitas

0e0

1e10

9E9

1.5e3

12.25E-2

2026.75e+10

Rejeitadas

e10

1e

1.e3

.5e2

01e2

1.2e3.4

Casos-limite: 0e0 (menor cadeia aceita) e 1e (expoente sem dígito).

AFNε de Thompson

42 estados · 52 transições (40 ε, 12 rotuladas) · inicial q0 · final q39

Literal de string

Alfabeto Σ

$\Sigma = C \cup \{ " \}$ · $C = L \cup D \cup \{ \text{espaço } _ . , ! ? : ; + - * / = < > \}$

Linguagem L

Aspas, zero ou mais símbolos de C e aspas. Sem escapes: aspas internas, `\`, tabulação e quebra de linha ficam fora.

ER formal

" C* "

Sintaxe no código (REGEX_ER05)

"[A-Za-z0-9 _.,!?:;+*/=<>-]*"

Equivalência

$[...]* \equiv C^*$ · dentro da classe, `.` e `*` são literais · - no fim da classe é literal · " não pertence a C

Aceitas

" "

"Andrey"

"abc 123"

"a_b.c"

"hello!"

"12:30"

Rejeitadas

ϵ

"abc

abc"

"a"b"

"a\b"

"linha + ↵"

Casos-limite: "" (conteúdo vazio, aceito) e ϵ (nem as aspas). ↵ = quebra de linha.

AFN ϵ de Thompson

8 estados · 9 transições (6 ϵ , 3 rotuladas) · inicial q0 · final q7

Literal de horário

Alfabeto Σ

$\Sigma = D \cup Q \cdot D = \{0, \dots, 9\} \cdot Q = \{:\}$

Linguagem L

Horários HH:MM com exatamente dois dígitos de cada lado, HH de 00 a 23 e MM de 00 a 59.

ER formal

$((0|1)D \mid 2(0|1|2|3)) Q (0|1|2|3|4|5) D$

Sintaxe no código (REGEX_ER06)

$([01][0-9] \mid 2[0-3]):[0-5][0-9]$

Equivalência

$[01] \equiv (0|1) \cdot [0-3] \equiv (0|1|2|3) \cdot [0-5] \equiv (0|\dots|5) \cdot [0-9] \equiv D \cdot$ a união separa 00–19 de 20–23

Aceitas

00:00 09:05 12:30 14:59 23:00 23:59

Rejeitadas

24:00 12:60 2:30 14:5 99:99 14:30:00

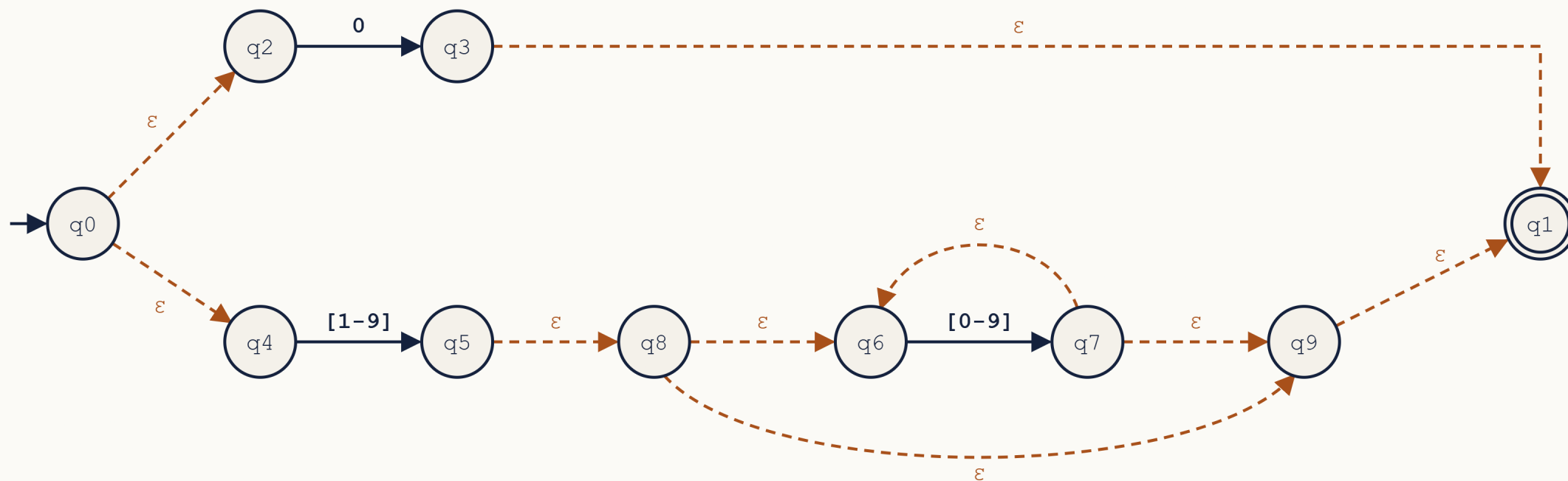
Casos-limite: 00:00 e 23:59 (extremos aceitos), 24:00 e 12:60 (primeiros valores fora).

AFNε de Thompson

16 estados · 16 transições (9 ε, 7 rotuladas) · inicial q0 · final q15

Construção de Thompson do inteiro

linha contínua = símbolo · tracejada = ϵ
inicial q0 · final q1 (círculo duplo)



Fecho-ε passo a passo

Cadeia 10 → aceita

Lê	Conjunto de estados
início	$E(q_0) = \{q_0, q_2, q_4\}$
1	$q_4 \rightarrow q_5; E = \{q_5, q_8, q_6, q_9, q_1\}$
0	$q_6 \rightarrow q_7; E = \{q_7, q_6, q_9, q_1\}$

Fim da cadeia com q1 no conjunto: aceita.

Cadeia 01 → rejeitada

Lê	Conjunto de estados
início	$E(q_0) = \{q_0, q_2, q_4\}$
0	$q_2 \rightarrow q_3; E = \{q_3, q_1\}$
1	nenhuma transição: $\emptyset$

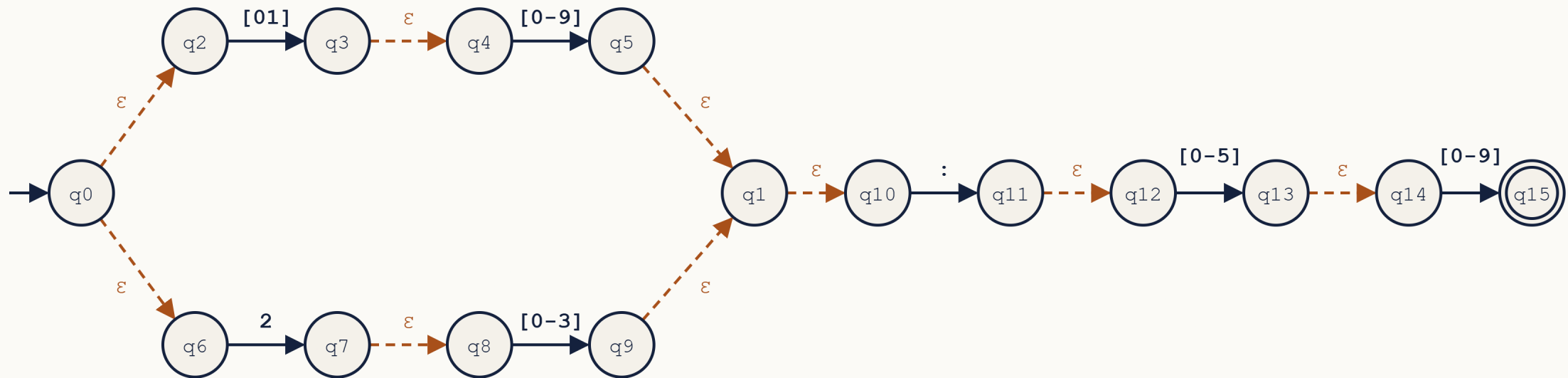
Conjunto vazio antes do fim: rejeitada. O zero só é aceito sozinho.

O simulador em src/automata.py faz exatamente isso e é comparado com re.fullmatch em todas as cadeias de 0 a 5 dígitos: 111.111 cadeias, 0 divergências.

AFN_ε · ER-06 · ([01][0-9]|2[0-3]):[0-5][0-9]

Construção de Thompson do horário

linha contínua = símbolo · tracejada = ε
inicial q0 · final q15 (círculo duplo)



Três camadas de verificação

129

testes pytest passando: ERs, AFNe, lexer, exemplos e CLI

0

divergências entre AFNe e re.fullmatch nas buscas exaustivas

6/6

AFNe idênticos à tabela canônica, em Python e no JFLAP

ER	Casos oficiais	Cadeias exaustivas	Divergências
ER-01	6 aceitas · 6 rejeitadas	3.906	0
ER-02	8 aceitas · 7 rejeitadas	111.111	0
ER-03	6 aceitas · 6 rejeitadas	364	0
ER-04	6 aceitas · 6 rejeitadas	19.608	0
ER-05	6 aceitas · 6 rejeitadas	19.531	0
ER-06	6 aceitas · 6 rejeitadas	177.156	0

DEMONSTRAÇÃO

O programa em execução

Cadeia aceita

```
python -m src.main examples/valido.min
```

Lista de tokens com tipo, lexema, linha e coluna.

Cadeia rejeitada

```
python -m src.main examples/invalido.min
```

ERRO LÉXICO · linha 2 · coluna 18 · lexema 007 · zero à esquerda não é permitido

Entrada do professor

```
python -m src.main
```

Digita-se uma linha de MiniLang; entrada vazia gera mensagem própria.

Validação das ERs

```
python tests/validate_all.py
```

```
python -m pytest
```

AFNε × regex e a suíte completa.

Limitações e possíveis melhorias

Limitações

- Somente análise léxica: não há sintaxe nem semântica
- Para no primeiro erro, sem recuperação
- Strings sem escape e sem acentos: "olá" é rejeitada
- Horário exige dois dígitos: 9:30 é rejeitado
- Números validam o formato, não o valor: 1e999 é aceito
- Testes exaustivos cobrem alfabetos reduzidos, não todo Σ^*

Melhorias

- Recuperação de erros para listar todos de uma vez
- Escapes em strings: " C* " vira " (C | \" | \\)* "
- Horário flexível: [01]?[0-9] na primeira alternativa
- Conversão AFN ϵ $\rightarrow$ AFD por construção de subconjuntos
- Analisador sintático sobre os tokens gerados

EQUIPE

Contribuições e uso de IA

Andrey Lourival Andrade Garcia

Especificação da MiniLang, AFNε de Thompson e simulador, MiniLexer, testes e validação AFNε × regex, diagramas JFLAP e Mermaid, README e fichas das ERs.

Igor Cecim Vilhena

Definição e formalização das Expressões Regulares: alfabetos, linguagens, notação formal e sintaxe Python. Relatório técnico e slides.

Uso de IA

Claude e ChatGPT usados como apoio em revisão, testes, diagramas e documentação. Declarado em DECLARACAO_IA.md; a equipe revisou e responde por todo o conteúdo.